Real time Operating Systems for Robot Control

Roman Aguilera

1 Problem/Objective

The demonstration of robotic agility is very common in robotics research. But despite this, standardization and definitions of robotic agility are non-existent or primitive at the very least. This work is a necessary technical step in defining such definitions. Efforts in defining agility can be done through experimental robotic simulations, but those assume that we can readily observe a robots immediate state (e.g. global position/velocity of a robot, position/velocity of robot's actuators), which is not a trivial task in the deployment of real world physical robots.

We wish to build a real-time robotic system from an RC car to implement preliminary agility research. Our purpose is to do intentional sliding and skidding to get an RC car to quickly reach a certain position and orientation. But in order to do that, we must first be able to sense and query the position and orientation of a robot in real-time. Once that can be done, our robot can readily query and use that information in a closed loop control scheme. To obtain these information points, we will mount an RGB-D camera [1] above our RC car in an experiment room to immediately obtain position and orientations of our car. The real-time aspect our system will be addressed using Real-Time Operating Systems (RTOSs), which allow a programmer to create software solutions with a high degree of reliability in timing [17].

The contributions of this project are: (i) an overview of Real-Time Operating Systems, (ii) a survey of Hard Real-Time Operating Systems , and (iii) a tutorial for configuring and compiling a Xenomai Cobalt (dual-kernel) kernel that can fully preempt the Linux-4.4.71 kernel. Section 4: "Findings" includes recommendations for someone interested in using Real-Time Operating Systems for hard real-time applications. This project is hosted on https://github.com/roman-aguilera/RTOS4ROBOTS.

2 Contributions

2.1 Overview of Real-Time Operating Systems

Real-Time Operating Systems offer deterministic timing guarantees for time critical applications. A common misconception is that Real-Time Operating Systems make programs run faster. The actual purpose of real-time operating

systems is to ensure that a high-priority task will be started and completed within a specified time window [9]. As a result, tasks of lesser priority will be ignored if there is a critical task that requires attention.

2.1.1 Definitions of "Real-Time"

To be able to apply real-time operating systems, one must first know the appropriate definitions that constitute a system as "real-time". There are three categories that real-time systems fall into: hard, firm, and, soft.

Hard Real-Time is used to define a system where the usefulness of a response to a critical-task request is zero after its deadline. Here, a program missing deadline constitutes catastrophic failure.

Firm Real-Time is used to define a system where infrequent deadline misses are tolerable, but may degrade the system's quality of service. Here, missing a deadline is very detrimental to quality of service.

Soft Real-Time is used to define a system where the usefulness of a result degrades after its deadline, thereby degrading the system's quality of service. Here, missing a deadline degrades the quality of service.

2.1.2 Relating Definitions of Real-Time to Real-Time Operating Systems

For our context, we focus on Hard Real-Time, as we care about technology that can provide the upmost reliability.

Tasks are software that is controlled by the the Real Time Operating System. RTOSs can activate, deactivate, terminate, and preempt tasks.

Handlers are software that are also controlled by the RTOS. They are executed by the interruption of software that is currently running. Handlers have higher priority than any other task.

Interrupts are when an operating system makes a critical call to the CPU and any currently running task is paused.

RTOS APIs are system calls. They are the programming functions that can be used by a task to execute RTOS functions on other tasks. The 3 most most common APIs for RTOSs are: Activate, Terminate, and Delay.

Latency is defined as the time between event and the systems response to that event. In the context of Hard Real-Time Operating Systems, consistency and predictability of interrupt latency [4] is of utmost importance. Lower interrupt latencies are obviously preferred, however it is more important to ensure a consistent interrupt latency.

Jitter is defined as the time deviation from desired signal time to actual signal time. Thus, jitter is a variation in latency. If the interrupt jitter exceeds a threshold, it is considered to be a complete system failure. Hard Real-Time Operating systems provide guarantees on maximum interrupt jitter for prioritized tasks, which inherently provides guarantees on maximum interrupt latency.

2.1.3 Task Execution Framework for Real-Time Operating Systems

Most Real-Time Operating Systems are preemptive. That means that the operating system has the capability to directly suppress low priority tasks when high priority tasks need to be executed [10]. RTOSs allow the programmer to manually set the priority level of tasks. Multiple tasks can have the same priority level. The tasks are controlled by a scheduler that typically uses a Priority Based First-Come-First-Serve (FCFS) algorithm. The FCFS scheduling algorithm places tasks that requests CPU computational resources on one of multiple queues. Each queue has an associated priority level, so if a task is of highest priority, it will be placed on the highest priority queue, and if a tasks is of second level priority, it will be placed on the second level priority queue, and so forth. The CPU services tasks on the highest priority queues in the order that they come in. Whenever there are no tasks in the highest priority queue, the CPU services tasks from the secondary-level priority queue in the order that they come in, and so forth.

2.1.4 Architectures of Real-Time Operating Systems

Approaches in the design of real-time operating systems fall into two broad categories: single-kernel approaches and dual-kernel approaches.

Single-Kernel Real-Time Operating Systems are ones that make the entire operating system preemptive. They aim at turning turning the entire kernel itself into a real-time kernel, capable of providing real-time guarantees.

Dual-Kernel Real-Time Operating Systems use a general operating system kernel and pair it side by side with a secondary real-time kernel. The secondary real-time kernel is the one that takes higher priority over the regular kernel and provides real time guarantees.

2.2 Survey of Real-Time Operating System Solutions

We narrowed our survey to hard Real-Time Operating Systems that are relevant to our end goal of implementing real-time robot control.

Xenomai: A free open-source hard RTOS, it provides real-time to user-space applications [21], and prioritizes clean extensibility as well as seamless integration with the Linux environment [3]. It's latencies are comparable to industrial-grade RTOSs. It supports ARM, ARM64, PPC32, x86, x86_64.

RTAI: It is a free open-source hard RTOS, boasting some of the lowest latencies. It supports x86, x86_64, PowerPC, ARM, m68k [16] [18]. A server-side side project, RTAI-XML, through a TCP/IP network using an XML protocol. It was made to allow students to implement real-time closed-loop control applications on target hardware without having to worry about breaking the host-side real-time code [14] [15]. Xenomai and RTAI were originally the same project but diverged because Xenomai stresses extensibility rather than lowest latencies [13].

FreeRTOS: An open source hard RTOS, the FreeRTOS Kernel consists of only 3 C files. The project is well documented and supported [19]. Users

of FreeRTOS can opt-in to switch to a proprietary option in order to receive the benefit of enterprise-level safety guarantees. With no advanced options like driver support or networking, FreeRTOS is more of a thread library rather than an operating system [2].

Real-Time Linux: Real-Time Linux is Linux's attempt at meeting real-time guarantees. The large community base of Linux is appealing for using Real-time Linux but, it cannot it cannot formally provide hard real-time guarantees. Side-by-side tests comparing implementations of Real-Time Linux vs Xenomai have found Xenomai with smaller standard deviations in latencies. [22] [8]. A similar experiment was conducted for one case on an Altera Cyclone V System-on-Chip board (based on an ARM Cortex A9 platform) [5]. This experiment found that Xenomai and Linux had the same Worst-Case Execution Time (WCET) for a user space task, with Xenomai exhibiting less execution time variability [6].

RT-Linux: Not to be confused with Real-Time Linux, RTLinux is a proprietary dual-kernel hard RTOS that runs linux as a fully preemptive process. It is owned by FSM Labs and Wind River Systems [20] .

RedHawk Linux: A proprietary single-kernel RTOS owned by Concurrent, it supports x86 and ARM processors [11].

QNX Neutrino: An enterprise-grade hard RTOS owned by Blackberry, QNX Neutrino is engineered under the single-kernel paradigm. Processors supported include x86, ARM, XScale, PowerPC, MIPS, and SH-4 [12].

Kithara Real-Time Suite: Kithara is a proprietary, real-time extension to Windows. It has has comprehensive driver and programming support for automation solutions. Hard real-time performance is provided when ran under "kernel mode". It can be used free of charge for 90 days [7].

2.3 Configuring and Compiling the Xenomai Kernel

For those interested in quickly compiling a Xenomai Cobalt (dual-kernel) with Linux-4.4.71 for x86_64 architectures, see the README.md file in the github link https://github.com/roman-aguilera/RTOS4ROBOTS.

3 Implementation

We chose to use the Intel RealSense RGBD xamera because it is a popular, relatively inexpensive, RGB-D camera for many computer vision research allocations. It runs on an x86 architecture, and the Software Development Kit is only compatible with Linux versions 4.4, 4.8, 4.10, 4.13, 4.15, 4.18 5.0 and 5.3. We plan to run out entire control system scheme from our personal computer which is based on x86 [23]. We chose to use Xenomai because it is open-source, and integrates well with Linux.

Installing Xenomai is relatively straight forward for most architectures, but x86 architectures require more steps. A large portion of time was spent on installing Xenomai, with a significant amount of time (about 3 weeks) spent

on learning about specialized kernel configurations and troubleshooting kernel compilation issues. It required careful bookkeeping and trial-and-error. Xenomai requires certain kernel configuration options to be enabled and others to be disabled. We attempted to configure 2 different kernels for Xenomai before we were able to even reach the kernel compilation process on the third kernel. One reason for difficulties was because of dependencies resulting in an inability to directly enable/disable some kernel configuration options. More specifically, for some Linux kernel versions, certain kernel configuration options could not be disabled due to them being automatically enabled/disabled when another kernel configuration option was enabled/disabled. These dependencies were sometimes non-intuitive. Further adding to complications, one must check whether certain configuration options that are found in one kernel are found in a different location for another kernel's configuration, or whether the options just do not exist for that other kernel. Another dilemma faced was dealing with the additional Xenomai kernel configuration options patched on top of the standard standard Linux ones, with limited relevant online resources to help.

4 Findings

We have successfully completed the, long and error-prone, process of configuring and compiling of a Xenomai dual-kernel to be run on x86_64 based embedded systems. We are now that much closer to creating a physical experiment design for our research on reliable and agile systems. We hope to run Xenomai kernel on and RC car's hardware in order to perform reliable skidding maneuvers.

With limited community support, dual-kernel approaches for Real-Time Operating Systems require a user to address a lot of maintenance issues. However, they are the best option when looking to create state-of the art real-time system using open source real-time operating systems. Therefore, to prevent wasted time on design efforts, it is imperative that a real-time system designer ask themselves some critical questions before considering the use of real-time operating systems. Questions include: (i) What is the maximum tolerable latency and jitter (microseconds, or milliseconds) for my application? (ii) Is my real-time application critical enough that missing timing guarantees would lead to a catastrophe? (iii) Is my hardware capable of providing hard real-time speeds?

For applications that require hard real-time guarantees, the best open source hard Real-Time Operating Systems would be Xenomai or RTAI. There are some notable proprietary hard RTOSs to consider that hand out free educational licenses: Blackberry's QNX Neutrino, and WindRiver's VxWorks. Kithara's Real-Time Suite, a real-time extension of windows, provides comprehensive support and is available for free for 90 days to anyone [7]. Linux's very own Real-Time Linux (not equivalent to RT-Linux) is currently on its way to becoming a hard RTOS, but it cannot provide hard real-time guarantees yet. It is worth looking into RTAI-XML, as it was specifically made to give users access to hard real-time hardware servers from a soft-real-time host [14] [15].

5 Related work

Similar work has been done to create an execution framework that aids in addressing real-time (performance, execution model, and practical) requirements for online reinforcement learning applications[25]. Ogma AI has created an RC car that can conduct online reinforcement learning. Training examples are given from a user controlling the RC car through a radio controller. The RC car uses a raspberry pi [24].

6 Consulting an Industry Professional

We consulted Niraj Raninga, a firmware engineer at Apple with experience in developing robotic control solutions. We learned that the usual process for developing a hardware system solution to a problem for a specific use case would be to create a preliminary system in which all of its components have top of the line specifications. The preliminary system is tested at each interface to ensure that use case requirements for latency and jitter are met and then the overall system undergoes latency and jitter tests. Then, if the system meets use case requirements, the next iterations of design have the hardware downgraded in order to find the cheapest possible final solution while still meeting use case specifications. His recommendations in hardware included (i) the use of a SOC (System-on-Chip) for control of the RC car, along with communication peripherals and (ii) the NVIDIA Jetson Nano for image processing as it is good for running multiple neural networks in parallel. We investigated the use of the Jetson for real-time applications. it turns out that The only hard RTOS that supports the NVIDIA Jetson is the RedHawk Linux RTOS. However, it is not an open source RTOS, and free licenses are not available to anyone.

References

[1] Depth camera d435i – intel® realsense™ depth and tracking cameras. https://www.intelrealsense.com/depth-camera-d435i/. (Accessed on 04/17/2020).

[2] Freertos open source licensing, freertos license description, freertos license terms and openrtos commercial licensing options. https://www.freertos.org/a00114.html#commercial.

[3] Home · wiki · xenomai / xenomai. https://gitlab.denx.de/Xenomai/xenomai/-/wikis/home.

[4] Interrupt latency. https://en.wikipedia.org/wiki/Interrupt_latency.

[5] Introduction to realtime linux. https://youtu.be/BKkX9WASfpI?t=1993.

[6] Introduction to realtime linux. https://youtu.be/BKkX9WASfpI?t=2395.

[7] Kithara realtime suite. =https://kithara.com/en/products/real-time-suite.

[8] Minimize jitter: Linux vs. xenomai. https://www.youtube.com/watch?v=tQ9tP-r8jx0.

[9] Myths and realities of real-time linux software systems. https://static.lwn.net/images/conf/rtlws11/papers/proc/p20.pdf.

[10] Preemption. https://en.wikibooks.org/wiki/Operating_System_Design/Scheduling_Processes/Preemption.

[11] Product of the day. https://www.linuxjournal.com/article/6645.

[12] Qnx neutrino realtime operating system (rtos). https://blackberry.qnx.com/en/software-solutions/embedded-software/industrial/qnx-neutrino-rtos.

[13] A quantitative comparison of realtime linux solutions: 3.2 connections and influences. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.136.4601&rep=rep1&type=pdf.

[14] Rtai-xml. =https://sites.google.com/site/rtaixml/rtaixml.

[15] Rtai-xml. https://sourceforge.net/projects/rtaixml/.

[16] the realtime application interface for linux. https://www.rtai.org/.

[17] What is a real-time operating system (rtos)? - national instruments. https://www.ni.com/en-us/innovations/white-papers/07/what-is-a-real-time-operating-system--rtos--.html. (Accessed on 04/17/2020).

[18] Rtai. https://en.wikipedia.org/wiki/RTAI, Jun 2019.

[19] Freertos. https://en.wikipedia.org/wiki/FreeRTOS, Mar 2020.

[20] Rtlinux. https://en.wikipedia.org/wiki/RTLinux, May 2020.

[21] Xenomai. https://en.wikipedia.org/wiki/Xenomai, Jan 2020.

[22] Jean-Luc Aufranc and Jean-Luc. A look at three options to develop real-time linux systems on application processors - hmp, real-time linux and xenomai. https://www.cnx-software.com/2016/10/15/a-look-at-three-options-to-develop-real-time-linux-systems-on-application-processors-hmp-real-time-linu Oct 2016.

[23] IntelRealSense. Intelrealsense/librealsense. https://github.com/IntelRealSense/librealsense/blob/master/doc/distribution_linux.md.

[24] Eric Laukien and Jamal. Self driving car learns online and on-board on raspberry pi 3. https://ogma.ai/2017/06/self-driving-car-learns-online-and-on-board-on-raspberry-pi-3/, Jun 2017.

[25] Robert Nishihara, Philipp Moritz, Stephanie Wang, Alexey Tumanov, William Paul, Johann Schleier-Smith, Richard Liaw, Michael I. Jordan, and Ion Stoica. Real-time machine learning: The missing pieces. *CoRR*, abs/1703.03924, 2017.